Rapport DAAR 2026

Boudrouss Réda
n°28712638

Sorbonne Université
France

Introduction

Contexte

Ce projet s'inscrit dans le cadre du cours DAAR 2026 et a pour objectif d'implémenter un système multi-agents de combat robotique selon le **modèle acteur**. Dans ce modèle, chaque agent est un processus isolé qui ne partage aucune mémoire avec ses voisins et ne communique que par envoi de messages asynchrones.

L'arène de jeu mesure 3000 × 2000 unités. Deux équipes s'y affrontent, chacune composée de cinq robots. Le simulateur fournit à chaque robot une interface de type `Brain` avec trois primitives de communication : `broadcast()` pour envoyer un message, `fetchAllMessages()` pour lire la boîte de réception, et `detectRadar()` pour observer l'environnement local.

Organisation du code

Le paquet `algorithms.MyBots` contient quatre fichiers :

Fichier	Rôle
`Utils.java`	Structures de données : `Position`, `Bullet`, `BotState`, `Enemy`
`BaseBot.java`	Classe abstraite : navigation, évitement, évasion de balles, communication
`MainBot.java`	Robot tireur (vitesse 1, PV 300) : détection, ciblage, tir
`AssistBot.java`	Robot éclaireur (vitesse 3, PV 100) : reconnaissance, relai d'information

Composition d'une équipe

Chaque équipe aligne cinq robots répartis en deux rôles :

- **3 MainBots** (MAIN1, MAIN2, MAIN3) positionnés en colonne centrale, chargés de tirer.
- **2 AssistBots** (NBOT, SBOT) positionnés en flanc nord et sud, chargés de scouter et de retransmettre les positions ennemies.

Aucun robot ne connaît l'état interne d'un autre. Toute coordination passe par des messages broadcastés dans la boîte aux lettres du simulateur.

Architecture

Hiérarchie des classes

Le simulateur expose une classe `Brain` que chaque robot doit étendre. `BaseBot` centralise la logique commune et délègue deux méthodes abstraites à ses sous-classes : `scanRadar()` et l'initialisation via `activate()`.

Machine à états finie

Chaque robot est gouverné par une machine à états finie à sept états définis dans `BaseBot.State`. Les transitions sont déclenchées par les résultats du radar ou l'arrivée de messages.

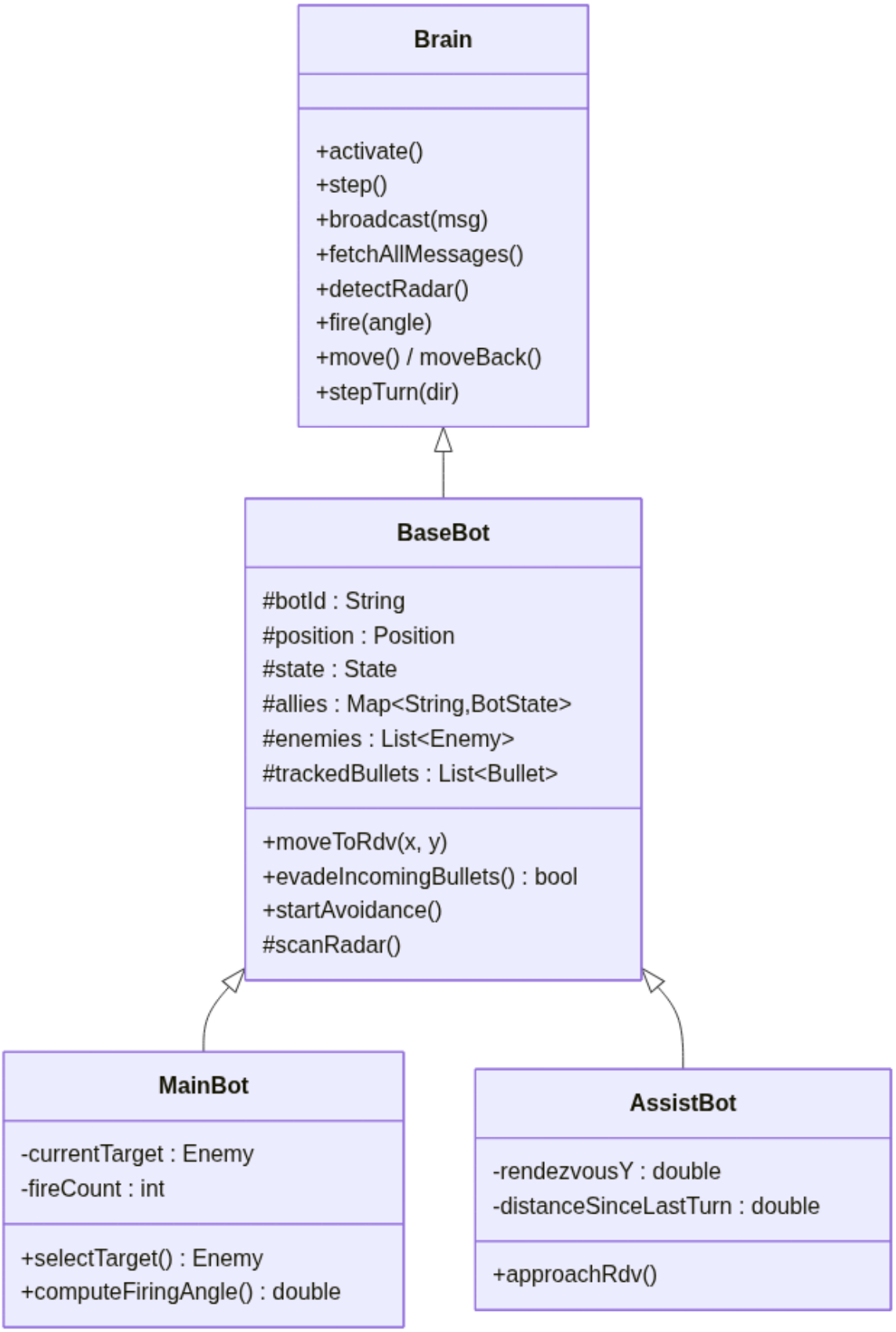

Figure 1 – Hiérarchie des classes

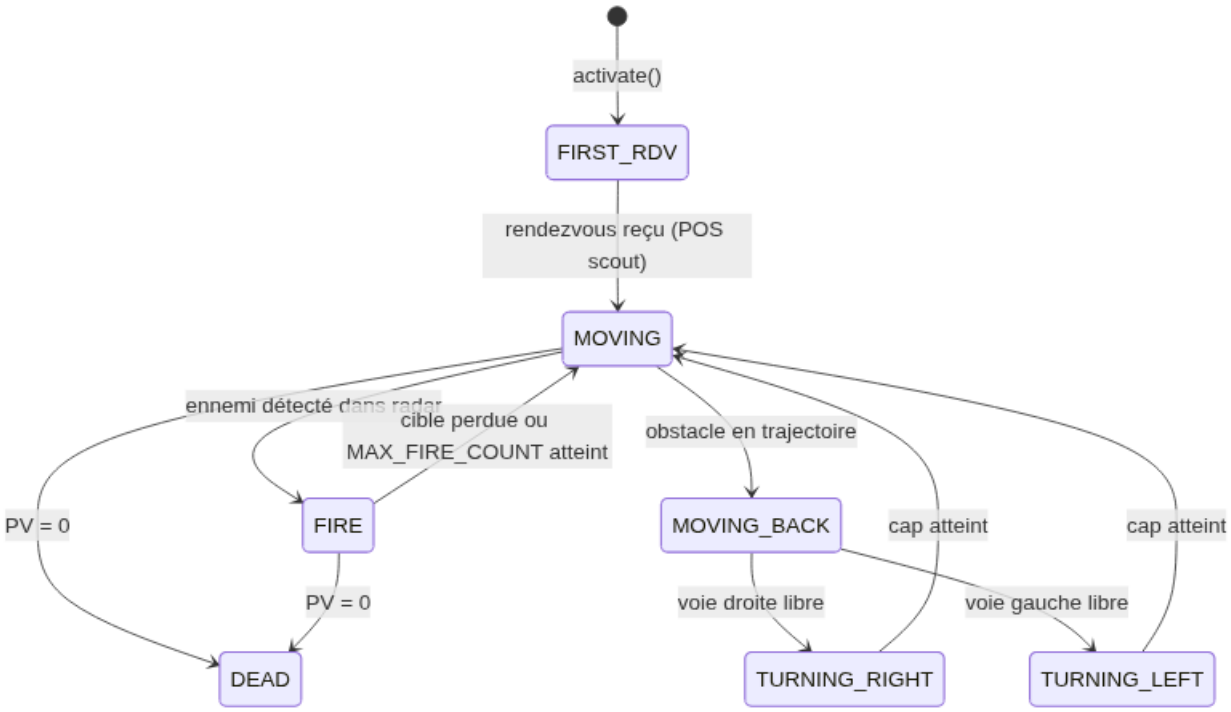

Figure 2 – Machine à états finie (BaseBot)

L'état `FIRE` est prioritaire : dès qu'un ennemi est sélectionnable, `MainBot.step()` force la transition vers `FIRE` avant toute autre évaluation.

Structures de données

`Utils.java` définit quatre classes utilisées comme conteneurs d'état.

`Position` stocke des coordonnées `(x, y)` en double et implémente `equals()` et `hashCode()` pour utilisation dans des collections.

`Bullet` représente un projectile détecté : position courante `(x, y)` et vitesse `(vx, vy)`. Le champ `hasVelocity` indique si la vitesse a déjà été calculée à partir de deux observations consécutives.

`BotState` maintient la position et le statut vivant/mort d'un allié. Elle est stockée dans la `Map<String, BotState> allies` indexée par identifiant de bot.

`Enemy` conserve un historique circulaire des trois dernières positions observées (`xHistory[0..2]`, `yHistory[0..2]`). La vitesse est calculée par différence finie : `speedX = xHistory[0] - xHistory[1]`. La méthode `predictPosition(t)` projette la position future en détectant d'abord une éventuelle oscillation (changement de signe de la vitesse entre deux pas consécutifs), auquel cas elle revient à la moyenne des deux dernières positions.

Navigation et évitement

Navigation par waypoints

La méthode `moveToRdv(tX, tY)` calcule l'angle vers la cible puis le restreint aux quatre directions cardinales via `getNearestAllowedDirection()`. Cette contrainte est appliquée aux

MainBots pour garantir des trajectoires rectilignes prévisibles et faciliter la vérification de dégagement de ligne de tir. Les AssistBots ignorent cette restriction et naviguent librement.

À chaque pas, `tryMove(forward)` calcule la position prédite après déplacement et appelle `isWithinBounds()` avant d'exécuter le mouvement. La marge de sécurité est de 100 unités pour les MainBots, 150 pour les AssistBots. Si la position prédite sort des limites, `startAvoidance()` est déclenché immédiatement.

Évitement d'obstacles

`startAvoidance()` teste trois chemins alternatifs dans l'ordre de priorité suivant :

1. Tournant à droite : cap courant + 0.5π
2. Tournant à gauche : cap courant − 0.5π
3. Recul : cap inverse

Pour chaque candidat, la méthode itère sur tous les obstacles détectés par le radar et teste si leurs coins (bounding box de rayon `r`) tombent dans le couloir de trajectoire via `isPointInTrajectory()`.

`isPointInTrajectory()` construit un rectangle de largeur `2 × BOT_RADIUS` et de longueur `2 × BOT_RADIUS` centré sur le cap à tester, puis délègue à `isPointInRectangle()`. Ce dernier utilise deux projections par produit scalaire pour tester l'appartenance à un rectangle quelconque orienté :

$$0 \leq \overrightarrow{AP} \cdot \overrightarrow{AB} \leq |\overrightarrow{AB}|^2 \quad \text{et} \quad 0 \leq \overrightarrow{AP} \cdot \overrightarrow{AD} \leq |\overrightarrow{AD}|^2$$

Le premier chemin sans obstacle retenu détermine le nouvel état (`TURNING_RIGHT`, `TURNING_LEFT` ou `MOVING_BACK`) et la cible d'évitement (`avoidTargetX, avoidTargetY`) placée à 50 unités dans la direction choisie.

Pour l'`AssistBot`, `avoidanceFraction` est tiré aléatoirement dans `[0.2, 0.8]` à chaque évitement, ce qui varie l'angle de dégagement entre 36° et 144°.

Évasion de balles

L'évasion repose sur trois méthodes chaînées dans `BaseBot`.

`updateTrackedBullets(rawBullets)` associe chaque balle détectée par le radar à son observation précédente. La correspondance utilise la position prédite (`x + vx, y + vy`) avec une tolérance de 15 unités (légèrement supérieure à la vitesse de 10 unités/pas pour absorber les erreurs d'arrondi). La vitesse (`vx, vy`) est calculée par différence entre la position courante et celle du pas précédent.

`isBulletThreat(b, px, py, danger)` calcule la distance perpendiculaire du point (`px, py`) à la trajectoire de la balle par la formule :

$$d_\perp = \left| \overrightarrow{dx} \cdot \hat{v}_y - \overrightarrow{dy} \cdot \hat{v}_x \right|$$

où $(\hat{v}_x, \hat{v}_y)$ est le vecteur vitesse normalisé et (dx, dy) le vecteur du bot vers la balle. Une menace est confirmée si $d_\perp < r_{bot} + r_{balle} + 10$ et si la balle se dirige bien vers le bot (produit scalaire positif).

`performDodge()` anticipe 5 pas en avant et 5 pas en arrière le long du cap courant, et exécute le déplacement qui sort le bot de la zone de danger. En l'absence des deux options, le recul est appliqué par défaut.

Détection et tir

Détection et suivi des ennemis

`MainBot.scanRadar()` convertit chaque résultat radar en coordonnées absolues :

$$x_{ennemi} = x_{bot} + d \cdot \cos(\theta), \quad y_{ennemi} = y_{bot} + d \cdot \sin(\theta)$$

Les ennemis détectés sont transmis via `broadcast("ENEMY ...")` pour que les alliés puissent les enregistrer sans les avoir vus directement.

`trackEnemy()` maintient la liste `enemies` par fusion : si un ennemi existe à moins de 50 unités de la position rapportée, sa fiche est mise à jour via `updatePosition()` ; sinon un nouvel `Enemy` est créé. Pour les positions reçues par message (et non par radar direct), la distance et la direction sont recalculées depuis la position courante du bot.

La classe `Enemy` détecte l'oscillation entre deux pas consécutifs en comparant le signe des deltas de position. Si `prevDx * currentDx < 0` ou `prevDy * currentDy < 0`, la cible est considérée en oscillation et la position prédite est ramenée à la moyenne des deux dernières positions au lieu d'extrapoler la vitesse.

Sélection de cible

`selectTarget()` trie la liste `enemies` par distance croissante, puis teste chaque ennemi jusqu'à en trouver un dont la ligne de tir est dégagée :

1. Prédire la position de l'ennemi au temps de vol $t = d/v_{balle}$.
2. Vérifier que cette position prédite n'est pas masquée par un allié via `isAllyInFireLine()`.
3. Retourner le premier ennemi qui passe ce filtre.

`isAllyInFireLine()` prédit également la position future des alliés : les scouts se déplacent à vitesse 3 si un MainBot est à moins de 1250 unités, sinon restent stationnaires ; les autres MainBots avancent d'une unité le long du cap courant.

Calcul de l'angle de tir

`computeFiringAngle()` opère en trois étapes.

Étape 1 : prédiction. La position future de la cible est calculée avec le temps de vol de la balle :

$$t = \frac{d}{v_{balle}}, \quad (x_f, y_f) = (x_0 + v_x \cdot t,\ y_0 + v_y \cdot t)$$

Étape 2 : angle central et déviation maximale. L'angle vers la position prédite est :

$$\theta_c = \arctan 2(y_f - y_{bot},\ x_f - x_{bot})$$

La déviation maximale admissible pour toucher une cible de rayon r_{bot} avec une balle de rayon r_{balle} est :

$$\Delta\theta = \arctan\left(\frac{r_{bot} + r_{balle} - 1}{d}\right)$$

Étape 3 : sélection de l'angle. Trois angles sont testés dans l'ordre $\{\theta_c,\ \theta_c - \Delta\theta,\ \theta_c + \Delta\theta\}$. Pour chaque angle, `isLineClear()` projette un segment jusqu'à la portée maximale de la balle (1000 unités) et vérifie que ni un allié ni une épave ne se trouve à moins de $r_{bot} + r_{balle}$ de ce segment. Le premier angle passant ce test est retenu.

Si aucun angle n'est valide, `computeFiringAngle()` retourne `NaN` et le bot revient en état `MOVING`.

Manoeuvre post-tir

Après chaque tir, `isManeuvering` est mis à vrai. `maneuverAfterFire()` exécute alors une repositionnement :

- Si le cap n'est pas aligné avec la cible (tolérance 0.5 rad) : rotation vers la cible.
- Si la distance à la cible est inférieure à 980 unités : recul.
- Sinon : avance.

Un compteur `fireCount` limite à 100 le nombre de tirs consécutifs sur la même cible. Au-delà, l'ennemi est retiré de la liste (il est probablement détruit ou hors portée) et le bot retourne en `MOVING`.

Modèle acteur et coordination

Le modèle acteur dans le simulateur

Chaque robot est un acteur au sens strict : il dispose d'un état privé opaque, ne partage aucune mémoire avec ses coéquipiers, et communique uniquement par messages. Le simulateur joue le rôle de boîte aux lettres : `broadcast(msg)` dépose un message dans la file de tous les bots de l'équipe, `fetchAllMessages()` vide la file du bot appelant.

Cette architecture élimine toute synchronisation par verrou. Deux robots peuvent lire et écrire simultanément sans risque de corruption, car le simulateur sérialise les appels par tick. La contrepartie est que la communication est asynchrone et non fiable dans le sens du modèle acteur : un message émis au tick t n'est disponible qu'au tick $t + 1$ au plus tôt.

Protocole de messages

Cinq types de messages sont définis sous forme de chaînes brutes non typées :

Type	Format	Émetteur	Usage
`POS`	`POS id x y heading`	Tous	Partager la position courante
`ENEMY`	`ENEMY dir dist type x y`	MainBot, AssistBot	Signaler un ennemi détecté
`WRECK`	`WRECK x y`	MainBot, AssistBot	Signaler une épave
`DEAD`	`DEAD id`	AssistBot	Annoncer sa propre destruction
`MOVING_BACK`	`MOVING_BACK id x y`	AssistBot	Signaler un ennemi trop proche

`POS` est émis à chaque appel de `tryMove()`, ce qui garantit une mise à jour de la carte alliée à chaque déplacement effectif. Les MainBots utilisent les positions des scouts (`SBOT` en priorité, `NBOT` si SBOT est mort) comme waypoint de ralliement dans leur état `FIRST_RDV` et `MOVING`.

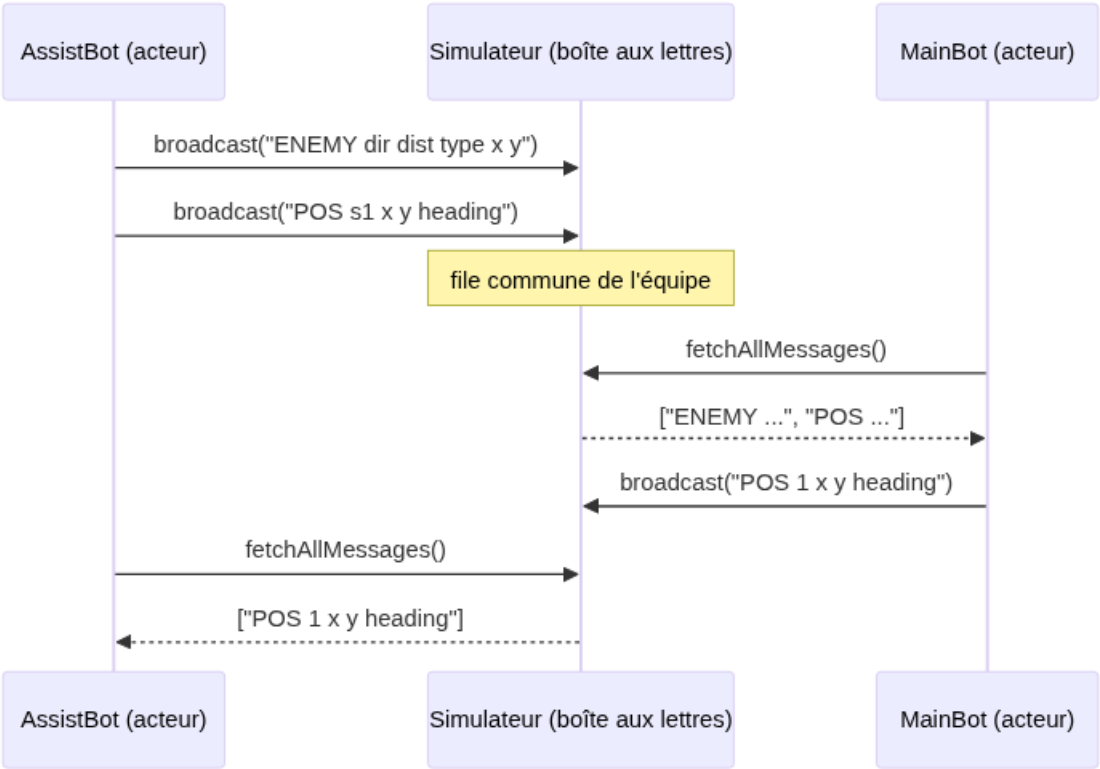

Figure 3 – Flux de messages entre acteurs (un tick)

Comportement de l'AssistBot

L'AssistBot a un rôle de capteur mobile. Son cycle de vie se décompose en deux grandes phases.

Phase d'approche (`isApproachingRdv = true`). Le bot exécute une séquence en trois étapes sans lire les messages : rotation vers le nord (NBOT) ou le sud (SBOT), déplacement vertical jusqu'à `rendezvousY` (780 ou 1250 unités), puis rotation vers l'est ou l'ouest selon l'équipe. Cette initialisation est purement locale et ne dépend d'aucun message.

Phase active. Le bot se déplace horizontalement et diffuse les ennemis détectés. Il se fige (`isFrozen = true`) dans deux cas : obstacle en trajectoire ou ennemi MainBot adverse à moins de `4 × BOT_RADIUS = 200` unités. Quand il est gelé, il émet `MOVING_BACK` et cesse tout déplacement jusqu'au prochain tick radar. L'évasion de balles reste active dans cet état.

Un compteur `distanceSinceLastTurn` déclenche un évitement après 1500 unités parcourues (moitié de la largeur d'arène), ce qui force périodiquement le scout à changer de cap et couvrir une zone différente.

Limites du modèle

L'implémentation présente plusieurs écarts par rapport à un modèle acteur complet :

- **Messages non typés.** Les messages sont des chaînes parsées manuellement par `split("  ")`. Une erreur de formatage provoque une `NumberFormatException` silencieuse.
- **Pas de garantie d'ordre.** `fetchAllMessages()` renvoie les messages dans l'ordre d'arrivée, mais rien ne garantit qu'un `DEAD` soit traité avant le `ENEMY` suivant du même bot.
- **Pas d'accusé de réception.** Un MainBot ne sait pas si son allié a bien reçu une position ennemie. La rediffusion systématique compense partiellement cette absence.
- **Identité auto-assignée.** Chaque bot détermine son propre identifiant au démarrage en scannant le radar (`determineIdentity()` pour MainBot, détection d'obstacle nord pour AssistBot). Ce mécanisme suppose une configuration initiale stable et échouerait si deux bots obtenaient le même identifiant.